

What is sorting → arrangement of data based on certain parameter.

1 2 3 5 7 ✓ (asc)

7 5 3 1 -1 ✓ (desc)

no. of factors →

1	7	2	9	24
↓	↓	↓	↓	↓
1	2	2	3	8

sorted in asc order by no. of factors.

Why sorting? → searching becomes easy

Name	Value
Ankit	10
Hannav	6
Rachana	6
Rachana	6
Hannav	6
Shubham	5
Sashwat	3
Subho	2

Sorted by the value in desc order

Sort by value if two people have same value sort by name

Name	Value
Ankit	10
Hannav	6
Rachana	6
Shubham	5
Sashwat	3
Subho	2

Name	Marks
Bikram	90
Airav	50
Anshul	60
Ashwin	90
Niladri	60
Utsav	0
Shaktiman	100
Praveen	50

sort the
data basis
on marks.
(asc)

Name	Marks
Utsav	0
Airav	50
Praveen	50
Anshul	60
Niladri	60
Bikram	90
Ashwin	90
Shaktiman	100

Stable Sort :

if two data pts have same value of parameter, then if we maintain their relative order in the sorted arr, then it is a stable sort.

Inplace Sorting : if the sorting algo. doesn't use any extra space, then it is in-place

Q1. Given an arr of size N, find kth smallest element

ex $\Rightarrow$ arr $\Rightarrow$ ~~1~~ 5 ~~4~~ 2 10 3

k = 3.

find 1st min, delete it

find 2nd min, delete it

find 3rd min return.

arr $\Rightarrow$

0	1	2	3	4	5
1	5	4	2	10	3

4	5	2	10	3
---	---	---	----	---

4	2	10	3
---	---	----	---

ans.

4	2	3	10
---	---	---	----

4	2	3	5	10
---	---	---	---	----

4	2	3	5	10
---	---	---	---	----

Selection

Sort

```

for ( i=0; i < N; i++) {
    min = arr[i];
    minI = i;
    for ( j= i+1; j < N; j++) {
        if ( arr[j] < min ) {
            min = arr[j];
            minI = j;
        }
    }
    swap ( arr[i], arr[minI] )
}

```

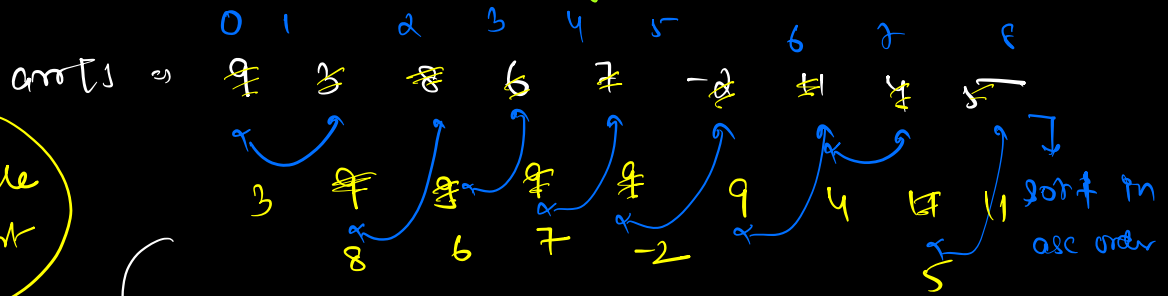
$TC \Rightarrow O(N^2)$
 $SC \Rightarrow O(1)$

* Stable ✓

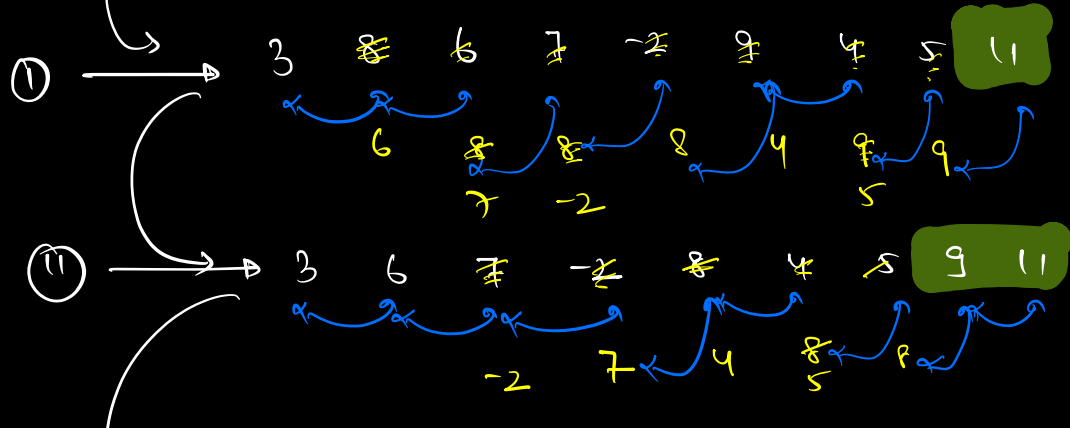
* Stable ✗

↓
 not stable
 bcz any
 element can
 go anywhere
 due to random
 swaps.

* only allowed to do adjacent comparisons.



Bubble sort



iii) $\rightarrow$ 3 6 -2 7 4 5 8 9 11

```
for (i=0; i<N; i++) {
```

```
    for (j=0; j<N-1; j++) {
```

```
        if (arr[j] > arr[j+1]) {
```

```
            swap(arr[j], arr[j+1])
```

```
        }
```

```
    }
```

```
}
```

$TC \Rightarrow O(N^2)$
 $SC \Rightarrow O(1)$

* in place $\rightarrow$ ✓

* stable $\rightarrow$ ✓

i/p $\rightarrow$ $\begin{array}{cccc} 2 & 2 & 2 & 1 \\ \hline 2 & 2 & 2 & 1 \\ \hline 2 & 2 & 2 & 1 \\ \hline 2 & 2 & 2 & 1 \\ \hline \end{array}$

sorted $\rightarrow$ $\begin{array}{cccc} 1 & 2 & 2 & 3 \\ \hline \end{array}$

Q Given 2 sorted arrays, merge 2 sorted arrays to get a final sorted array

A[] $\Rightarrow$ 2 5 7 12 20 24 29

B[] $\Rightarrow$ 6 9 10 14 15 19

O/p $\Rightarrow$ 2 5 6 7 9 10 12 14 15 19 20 24 29

Brute → merge both and sort →

Optimise
 $N \rightarrow A[] \Rightarrow 2 \quad 5 \quad 7 \quad 12 \quad 20 \quad 24 \quad 29$
 $M \rightarrow B[] \Rightarrow 6 \quad 9 \quad 10 \quad 14 \quad 15 \quad 19$

$Op[] \Rightarrow 2 \quad 5 \quad 6 \quad 7 \quad 9 \quad 10 \quad 12 \quad 14 \quad 15 \quad 19 \quad 20$
 $\uparrow$
 $(N+M)$ $24 \quad 29$

```

void merge (a[], b[], c[], int a.sizen, int b.sizem) {
    i=0; j=0; k=0;
    while (i < n & j < m) {
        if (a[i] <= b[j]) {
            c[k] = a[i];
            i++;
            k++;
        }
        else {
            c[k] = b[j];
            j++;
            k++;
        }
    }
}

```

```

while( i < N) {
    c[k] = a[i]
    i++
    k++
}
while( j < M) {
    c[k] = b[j]
    j++
    k++
}
}

```

a[] :	1	2	4	6	i
b[] :	0	2	5	8	j

c[] :	0	1	2	2	4	5	6	8
-------	---	---	---	---	---	---	---	---

$T \Rightarrow O(N+M)$
 $SC \Rightarrow O(N+M)$

→ Single array of N integers, and two parts are sorted

↓
 $l \rightarrow y-1$
 $y \rightarrow r$

Sort from $(l \rightarrow r)$

ex → i/p

0	1	2	3	4	5	6	7	8	9
8	1	3	6	11	2	4	9	7	6

$l=2, y=5, r=7$

o/p →

0	1	2	3	4	5	6	7	8	9
8	1	2	3	4	6	9	11	7	6

$l=2$
 $y=5$
 $r=7$

0	1	2	3	4	5	6	7	8	9
8	1	3	6	11	2	4	9	7	6

$\underline{l \rightarrow r}$
 (a, b)
 $\downarrow$
 $\underline{b-a+1}$

2	3	4	6	9	11
---	---	---	---	---	----

→

0	1	2	3	4	5	6	7	8	9
8	1	2	3	4	6	9	11	7	6


```

void merge (a[l], l, y, r) {
    c[l] = new int[r-l+1];
    i = l; j = y; k = 0;

```

← merge for
mergesort

```

    while (i < y & j < r) {
        if (a[i] <= a[j]) {
            c[k] = a[i];
            i++;
            k++;
        }
        else {
            c[k] = a[j];
            j++;
            k++;
        }
    }
}

```

```

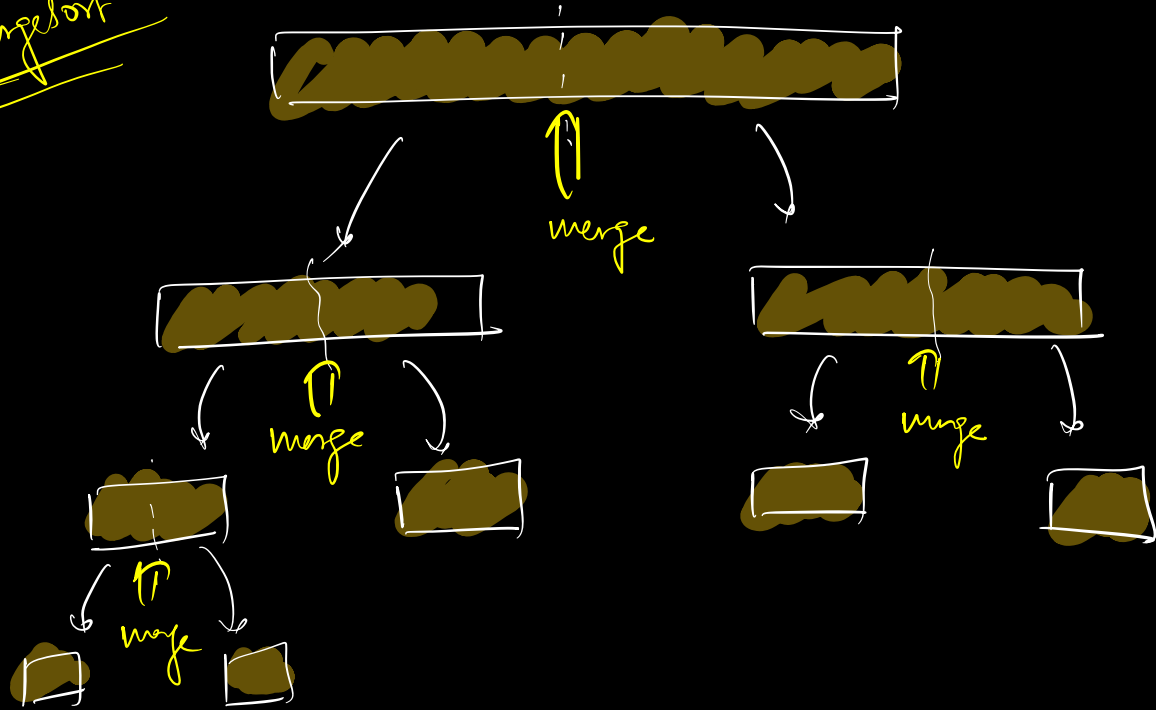
while (i < y) {
    c[k] = a[i];
    i++;
    k++;
}
while (j < r) {
    c[k] = a[j];
    j++;
    k++;
}
}

```

$TC \Rightarrow O(r-l)$ $SC \Rightarrow O(r-l)$
--

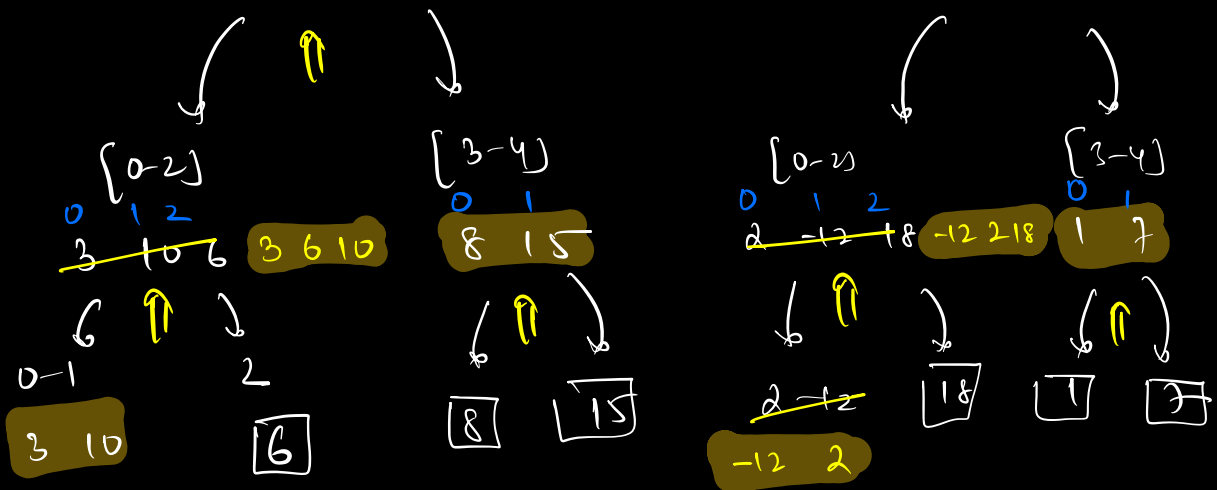
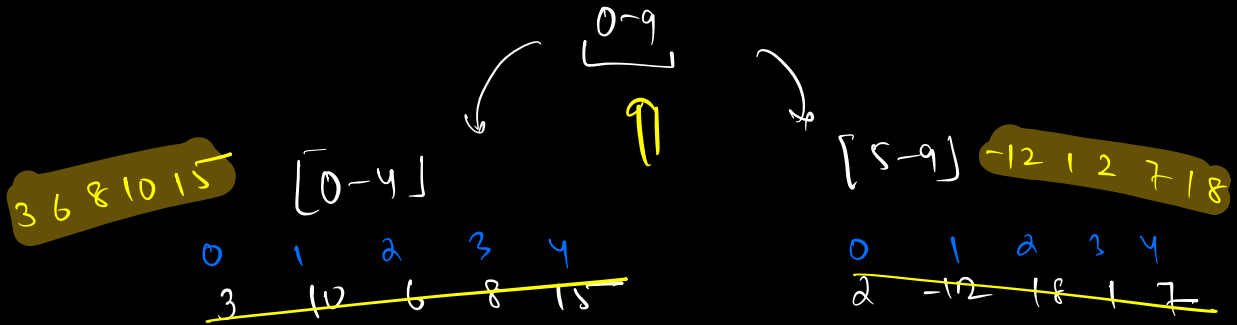
→ copy all elements from the newly sorted array to original array

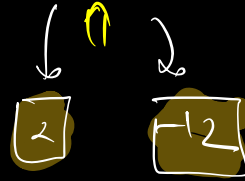
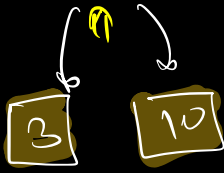
Merge sort



[-12 1 2 3 6 7 8 10 15 18]

0 1 2 3 4 5 6 7 8 9
3 10 6 8 15 2 -12 18 1 7





mergesort

```

void mergesort(arr, l, r) {
    if (l == r)
        return;

    int m = (l + r) / 2;

    mergesort(arr, l, m);
    mergesort(arr, m + 1, r);

    merge(arr, l, m + 1, r);
}

```

$\left\{ \begin{array}{l} l, m-1 \\ m, r \end{array} \right\}$
 $\left\{ \begin{array}{l} l, m \\ m+1, r \end{array} \right\}$

$$T(N) = 2T(N/2) + N$$

$$T(1) = 1$$

H.W

Calculate TC & SC

↓
next class